Rather than containing just a single `Location` and `String` message, we should get a `List[(Location,String)]`:

```
case class ParseError(stack: List[(Location,String)])
```

This is a stack of error messages indicating what the `Parser` was doing when it failed. We can now specify what `scope` does—if `run(p)(s)` is `Left(e1)`, then `run(scope(msg)(p))` is `Left(e2)`, where `e2.stack.head` will be `msg` and `e2.stack.tail` will be `e1`.

We can write helper functions later to make constructing and manipulating `ParseError` values more convenient, and to format them nicely for human consumption. For now, we just want to make sure it contains all the relevant information for error reporting, and it seems like `ParseError` will be sufficient for most purposes. Let's pick this as our concrete representation and remove the abstract type parameter from `Parsers`:

```
trait Parsers[Parser[+_]] {
  def run[A](p: Parser[A])(input: String): Either[ParseError,A]
  ...
}
```

Now we're giving the `Parsers` implementation all the information it needs to construct nice, hierarchical errors if it chooses. As users of the `Parsers` library, we'll judiciously sprinkle our grammar with `label` and `scope` calls that the `Parsers` implementation can use when constructing parse errors. Note that it would be perfectly reasonable for implementations of `Parsers` to not use the full power of `ParseError` and retain only basic information about the cause and location of errors.

9.5.3 Controlling branching and backtracking

There's one last concern regarding error reporting that we need to address. As we just discussed, when we have an error that occurs inside an `or` combinator, we need some way of determining which error(s) to report. We don't want to *only* have a global convention for this; we sometimes want to allow the programmer to control this choice. Let's look at a more concrete motivating example:

```
val spaces = " ".many
val p1 = scope("magic spell") {
  "abra" ** spaces ** "cadabra"
}
val p2 = scope("gibberish") {
  "abba" ** spaces ** "babba"
}
val p = p1 or p2
```

What `ParseError` would we like to get back from `run(p)("abra cAdabra")`? (Again, note the capital `A` in `cAdabra`.) Both branches of the `or` will produce errors on the input. The `"gibberish"`-labeled parser will report an error due to expecting the first word to be `"abba"`, and the `"magic spell"` parser will report an error due to the

accidental capitalization in `"cAdabra"`. Which of these errors do we want to report back to the user?

In this instance, we happen to want the `"magic spell"` parse error—after successfully parsing the `"abra"` word, we're *committed* to the `"magic spell"` branch of the `or`, which means if we encounter a parse error, we don't examine the next branch of the `or`. In other instances, we may want to allow the parser to consider the next branch of the `or`.

So it appears we need a primitive for letting the programmer indicate when to commit to a particular parsing branch. Recall that we loosely assigned `p1 or p2` to mean *try running p1 on the input, and then try running p2 on the same input if p1 fails.* We can change its meaning to *try running p1 on the input, and if it fails in an uncommitted state, try running p2 on the same input; otherwise, report the failure.* This is useful for more than just providing good error messages—it also improves efficiency by letting the implementation avoid examining lots of possible parsing branches.

One common solution to this problem is to have all parsers *commit by default* if they examine at least one character to produce a result.[12] We then introduce a combinator, `attempt`, which delays committing to a parse:

```
def attempt[A](p: Parser[A]): Parser[A]
```

It should satisfy something like this:[13]

```
attempt(p flatMap (_ => fail)) or p2 == p2
```

Here `fail` is a parser that always fails (we could introduce this as a primitive combinator if we like). That is, even if `p` fails midway through examining the input, `attempt` reverts the commit to that parse and allows `p2` to be run. The `attempt` combinator can be used whenever there's ambiguity in the grammar and multiple tokens may have to be examined before the ambiguity can be resolved and parsing can commit to a single branch. As an example, we might write this:

```
(attempt("abra" ** spaces ** "abra") ** "cadabra") or (
 "abra" ** spaces "cadabra!")
```

Suppose this parser is run on `"abra cadabra!"`—after parsing the first `"abra"`, we don't know whether to expect another `"abra"` (the first branch) or `"cadabra!"` (the second branch). By wrapping an `attempt` around `"abra" ** spaces ** "abra"`, we allow the second branch to be considered up until we've finished parsing the second `"abra"`, at which point we commit to that branch.

---

[12] See the chapter notes for more discussion of this.

[13] This is not quite an equality. Even though we want to run `p2` if the attempted parser fails, we may want `p2` to somehow incorporate the errors from both branches if it fails.